1. Requirement Formulation and Analysis

1.1 Sources Employed for Gathering Requirements

The requirements for the Court Case and Legal Document Management System were gathered from multiple sources to ensure comprehensive coverage of judicial workflows and data management needs.

1.1.1 Publicly Accessible Judicial Portals

Pakistani court websites such as the Supreme Court of Pakistan (supremecourt.gov.pk), Sindh High Court portal, and the National Judicial Policy Making Committee (NJPMC) case tracking system were studied. These portals provided insights into:

- Case registration formats (Civil, Criminal, Constitutional)
- Hearing scheduling and cause list structures
- Case status categories (Pending, Decided, Adjourned, Part-Heard)
- Document metadata formats used in judicial systems

1.1.2 Literature Review and Research

Published academic papers and existing DBMS-based court management system implementations were reviewed to understand common database schemas for legal systems, best practices for document management systems, security and access control requirements in judicial environments, and audit trail and record keeping standards.

1.1.3 Domain Analysis and Interviews

Conceptual interviews with individuals familiar with legal workflows (lawyers, law students) were conducted to identify key entities in court case management, understand the flow from case filing to judgment, identify document types and their relationships to cases, and understand the access levels required for different users.

1.2 Identified Data Requirements and Analysis

1.2.1 Key Entities Identified

Through analysis of the gathered requirements, the following core entities were identified:

Entity	Description	Key Attributes
Court	Represents a court of law	Court Name, Court ID, Court Type, City, Jurisdiction
Judge	Judges presiding over cases	Judge ID, Name(first, last), Court ID, Designation, Appointment Date
Case	Central entity - the legal case	Case ID, Case No, Case Type, Status, Filing Date, Status, Court ID, Judge ID, Description

Entity	Description	Key Attributes
Party	Persons involved in a case	Party ID, Full Name, Role, CNIC, Contact, Case ID
Lawyer	Legal representatives	Lawyer ID, Bar No, Full Name, Specialization, Contact, Email
Hearing	Court hearing sessions	Hearing ID, Hearing Date, Hearing Time, Court Room, Next ID, Case ID, Judge ID, outcome
Document	Legal documents per case	Doc ID, Doc Title, Doc Type, Upload Date, Upload by, File Path, Case ID
Judgment	Final verdicts on cases	Judgment ID, Judgement Date, Summary, Decision, Judge ID, Case ID
User	System users (staff)	User ID, Username, Role, Full Name, Password Hash, Created At, Email
Representation (Weak ES)	Lawyer represents a party and a case has representation	Rep no, Lawyer ID, Party ID, Case ID

1.2.2 Relationships Identified

- A Court has many Judges and handles many Cases
- A Case involves multiple Parties (Plaintiff/Defendant) and multiple Lawyers
- A Case has multiple Hearings and multiple Documents
- A Case results in at most one Judgment
- A Lawyer represents one or more Parties across Cases
- A Hearing is presided over by a Judge
- Users can manage Cases, Hearings, and Documents based on role

1.2.3 Functional Requirements

- Register new cases with all associated metadata
- Add, update, and search parties, lawyers, and judges
- Schedule and track hearings with room and judge assignments
- Upload and retrieve legal documents with metadata
- Record judgments and update case status accordingly
- Generate reports on active cases, hearing schedules, and decisions
- Role-based access: Lawyer, User

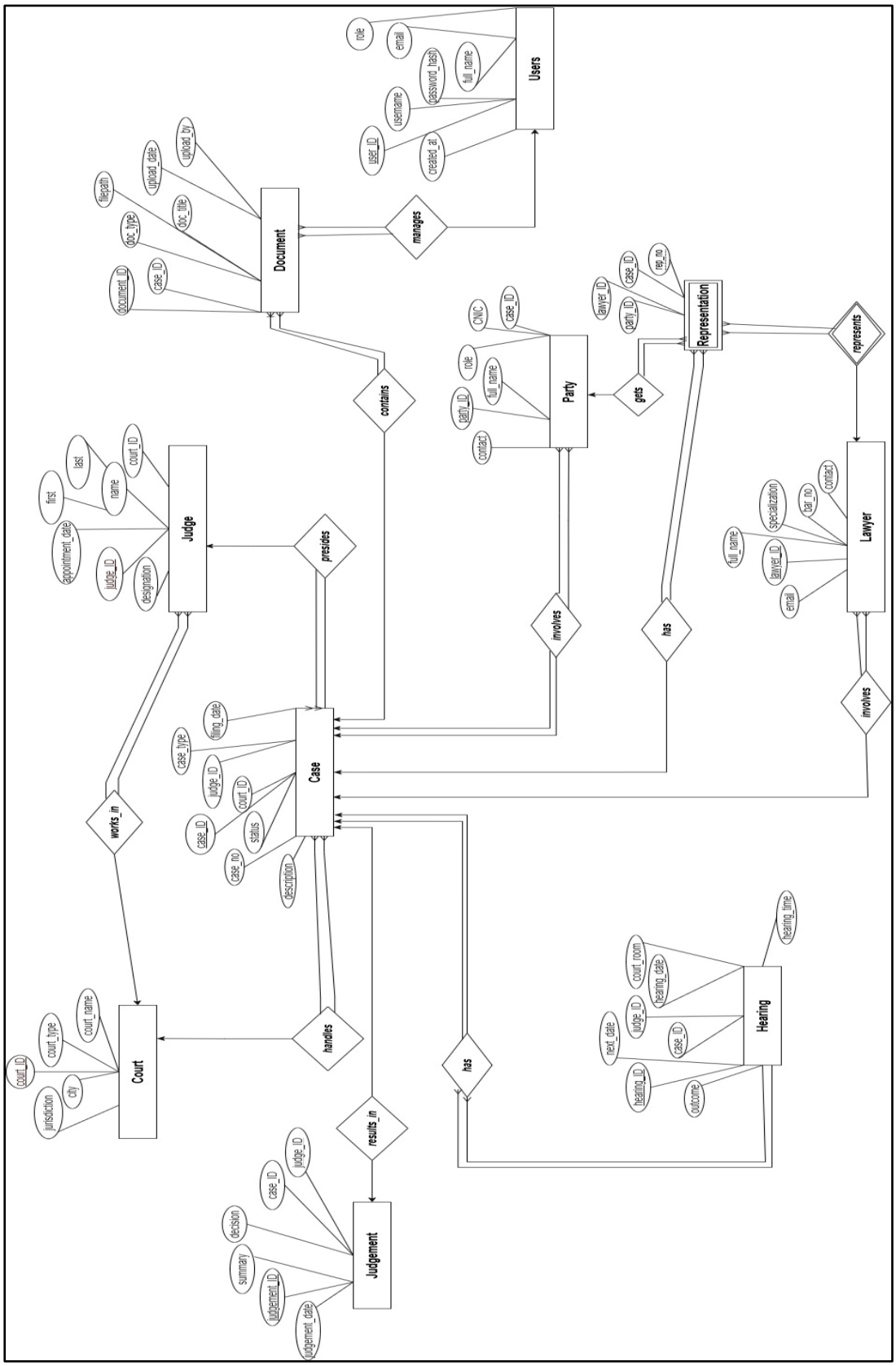
1.2.4 Non-Functional Requirements

- Data integrity enforced through primary and foreign keys
- Audit trails for all insertions and updates
- Role-based access control for sensitive case information
- Fast retrieval using indexed columns (Case No, Date, Party Name)
- Scalability to support thousands of cases and documents

2. ER Model of the Database

2.1 Entity-Relationship Diagram Description

The ER diagram models all entities, their attributes, and relationships. The following describes the complete ER model for the Court Case and Legal Document Management System.



2.2 Relationships Summary

Relationship	Cardinality	Participation	Foreign Key
Court -> Judge	1:N	Partial/Total	Judge.Court_ID
Court -> Case	1:N	Partial/Total	Case.Court_ID
Judge -> Case	1:N	Partial/Total	Case.Judge_ID
Case -> Party	1:N	Total/Total	Party.Case_ID
Case -> Hearing	1:N	Total/Total	Hearing.Case_ID
Case -> Document	1:N	Partial/Total	Document.Case_ID
Case -> Judgment	1:1	Partial/Partial	Judgment.Case_ID
Lawyer <-> Party (via REPRESENTATION)	M:N	Partial/Partial	Rep.Lawyer_ID, Rep.Party_ID
User -> Document	1:N	Partial/Total	Document.Uploaded_By

2.3 Transformation to Relational Tables

Each entity becomes a table. The M:N relationship between Lawyer and Party is resolved into the REPRESENTATION associative table. All foreign key references are explicit. Below is the list of final relational tables:

- COURT (Court_ID, Court_Name, Court_Type, City, Jurisdiction)
- JUDGE (Judge_ID, First_Name, Last_Name, Designation, Court_ID, Appointment_Date)
- CASE (Case_ID, Case_Number, Case_Title, Case_Type, Filing_Date, Status, Court_ID, Judge_ID, Description)
- PARTY (Party_ID, Full_Name, Role, CNIC, Contact_No, Address, Case_ID)
- LAWYER (Lawyer_ID, Bar_Number, Full_Name, Specialization, Contact_No, Email)
- REPRESENTATION (Rep_ID, Lawyer_ID, Party_ID, Case_ID)
- HEARING (Hearing_ID, Case_ID, Judge_ID, Hearing_Date, Hearing_Time, Courtroom, Outcome, Next_Date)
- DOCUMENT (Document_ID, Case_ID, Doc_Title, Doc_Type, Upload_Date, Uploaded_By, File_Path)
- JUDGMENT (Judgment_ID, Case_ID, Judge_ID, Judgment_Date, Decision, Summary)
- USERS (User_ID, Username, Password_Hash, Full_Name, Role, Email, Created_At)

3. Normalization of Tables

3.1 Overview

All tables in the database are analyzed and normalized up to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity.

3.2 First Normal Form (1NF)

A table is in 1NF if: all attributes are atomic (no repeating groups or multi-valued attributes), each row is unique (identified by a primary key), and all values in a column are of the same domain.

All tables satisfy 1NF. For example:

- PARTY: Each party has atomic values (Full_Name, CNIC, Contact_No are all single-valued). No repeating groups.
- HEARING: Hearing_Date and Hearing_Time are stored separately for atomicity.
- DOCUMENT: File_Path stores a single string path — no multi-valued attributes.
- REPRESENTATION: The M:N relationship is resolved into separate atomic rows, each identifying one lawyer-party-case combination.

3.3 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). 2NF is only relevant for tables with composite primary keys.

In our schema, all tables use single-column surrogate primary keys (e.g., Case_ID, Party_ID) with the exception of REPRESENTATION. Analysis:

- REPRESENTATION has a surrogate PK (Rep_ID). Although Lawyer_ID, Party_ID, and Case_ID are all foreign keys, there are no non-key attributes, so no partial dependency exists. The table is in 2NF.
- All other tables have a single PK, so 2NF is automatically satisfied.

3.4 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and has no transitive dependencies (no non-key attribute depends on another non-key attribute).

Analysis of each table:

Table	Potential Transitive Dep.	Resolution	3NF Status
COURT	None identified	N/A	In 3NF
JUDGE	Designation may imply Court hierarchy	Designation stored as freetext; Court_ID is FK	In 3NF
CASE	Status not derived from other non-key attr.	Status is an independent attribute	In 3NF
PARTY	None identified	N/A	In 3NF
LAWYER	None identified	N/A	In 3NF
REPRESENTATION	No non-key attributes	N/A	In 3NF

Table	Potential Transitive Dep.	Resolution	3NF Status
HEARING	Courtroom not dependent on Judge_ID	Stored independently from judge	In 3NF
DOCUMENT	None identified	N/A	In 3NF
JUDGMENT	Decision not derived from Summary	Both are independent	In 3NF
USERS	Role does not determine Email or Name	All attributes depend only on User_ID	In 3NF

Conclusion: All tables in the Court Case and Legal Document Management System database satisfy 1NF, 2NF, and 3NF. No further decomposition is required.

4. SQL Statements for Creation of Tables and Indices

4.1 Table Creation Statements (Oracle SQL)

4.1.1 COURT Table

```
CREATE TABLE COURT (     Court_ID      NUMBER(5)      CONSTRAINT pk_court PRIMARY KEY,
Court_Name    VARCHAR2(100)  NOT NULL,     Court_Type    VARCHAR2(50)   NOT NULL,     City
VARCHAR2(50)   NOT NULL,     Jurisdiction  VARCHAR2(100)  NOT NULL );
```

4.1.2 JUDGE Table

```
CREATE TABLE JUDGE (     Judge_ID          NUMBER(5)      CONSTRAINT pk_judge PRIMARY KEY,
First_Name        VARCHAR2(50)   NOT NULL,     Last_Name         VARCHAR2(50)   NOT NULL,
Designation       VARCHAR2(100)  NOT NULL,     Court_ID          NUMBER(5)      CONSTRAINT
fk_judge_court REFERENCES COURT(Court_ID),     Appointment_Date  DATE           NOT NULL );
```

4.1.3 USERS Table

```
CREATE TABLE USERS (     User_ID        NUMBER(5)      CONSTRAINT pk_users PRIMARY KEY,
Username       VARCHAR2(50)   CONSTRAINT uq_username UNIQUE NOT NULL,     Password_Hash
VARCHAR2(200)  NOT NULL,     Full_Name      VARCHAR2(150)  NOT NULL,     Role
VARCHAR2(30)   NOT NULL                     CONSTRAINT chk_role CHECK (Role IN
('Admin','Clerk','Lawyer','ReadOnly')),     Email          VARCHAR2(100)  CONSTRAINT uq_email
UNIQUE,     Created_At     DATE           DEFAULT SYSDATE );
```

4.1.4 CASE Table

```
CREATE TABLE CASE_INFO (     Case_ID      NUMBER(8)      CONSTRAINT pk_case PRIMARY KEY,
Case_Number  VARCHAR2(30)   CONSTRAINT uq_case_no UNIQUE NOT NULL,     Case_Title
VARCHAR2(200)  NOT NULL,     Case_Type    VARCHAR2(50)   NOT NULL                  CONSTRAINT
chk_case_type CHECK (Case_Type IN ('Civil','Criminal','Constitutional','Family','Labor')),
Filing_Date  DATE           NOT NULL,     Status       VARCHAR2(30)   NOT NULL
CONSTRAINT chk_status CHECK (Status IN ('Pending','Decided','Adjourned','Part-
Heard','Dismissed')),     Court_ID     NUMBER(5)      CONSTRAINT fk_case_court REFERENCES
COURT(Court_ID),     Judge_ID     NUMBER(5)      CONSTRAINT fk_case_judge REFERENCES
JUDGE(Judge_ID),     Description  VARCHAR2(500) );
```

4.1.5 PARTY Table

```
CREATE TABLE PARTY (     Party_ID    NUMBER(8)     CONSTRAINT pk_party PRIMARY KEY,
Full_Name   VARCHAR2(150) NOT NULL,     Role        VARCHAR2(30)  NOT NULL
CONSTRAINT chk_party_role CHECK (Role IN
('Plaintiff','Defendant','Petitioner','Respondent')),     CNIC        VARCHAR2(15)
CONSTRAINT uq_cnic UNIQUE,     Contact_No  VARCHAR2(20),     Address     VARCHAR2(300),
Case_ID     NUMBER(8)     CONSTRAINT fk_party_case REFERENCES CASE_INFO(Case_ID) );
```

4.1.6 LAWYER Table

```
CREATE TABLE LAWYER (     Lawyer_ID       NUMBER(8)     CONSTRAINT pk_lawyer PRIMARY KEY,
Bar_Number      VARCHAR2(20)  CONSTRAINT uq_bar UNIQUE NOT NULL,     Full_Name
VARCHAR2(150) NOT NULL,     Specialization  VARCHAR2(100),     Contact_No      VARCHAR2(20),
Email           VARCHAR2(100) );
```

4.1.7 REPRESENTATION Table

```
CREATE TABLE REPRESENTATION (     Rep_ID     NUMBER(8)  CONSTRAINT pk_rep PRIMARY KEY,
Lawyer_ID  NUMBER(8)  CONSTRAINT fk_rep_lawyer REFERENCES LAWYER(Lawyer_ID),     Party_ID
NUMBER(8)  CONSTRAINT fk_rep_party  REFERENCES PARTY(Party_ID),     Case_ID    NUMBER(8)
CONSTRAINT fk_rep_case   REFERENCES CASE_INFO(Case_ID),     CONSTRAINT uq_representation
UNIQUE (Lawyer_ID, Party_ID, Case_ID) );
```

4.1.8 HEARING Table

```
CREATE TABLE HEARING (     Hearing_ID    NUMBER(8)    CONSTRAINT pk_hearing PRIMARY KEY,
Case_ID       NUMBER(8)    CONSTRAINT fk_hear_case  REFERENCES CASE_INFO(Case_ID),
Judge_ID      NUMBER(5)    CONSTRAINT fk_hear_judge REFERENCES JUDGE(Judge_ID),
Hearing_Date  DATE         NOT NULL,     Hearing_Time  VARCHAR2(10) NOT NULL,     Courtroom
VARCHAR2(20) NOT NULL,     Outcome       VARCHAR2(200),     Next_Date     DATE );
```

4.1.9 DOCUMENT Table

```
CREATE TABLE DOCUMENT (     Document_ID  NUMBER(8)     CONSTRAINT pk_document PRIMARY KEY,
Case_ID      NUMBER(8)     CONSTRAINT fk_doc_case REFERENCES CASE_INFO(Case_ID),
Doc_Title    VARCHAR2(200) NOT NULL,     Doc_Type     VARCHAR2(50)  NOT NULL
CONSTRAINT chk_doc_type CHECK (Doc_Type IN
('FIR','Petition','Order','Verdict','Evidence','Affidavit','Bail Application','Other')),
Upload_Date  DATE          NOT NULL,     Uploaded_By  NUMBER(5)     CONSTRAINT fk_doc_user
REFERENCES USERS(User_ID),     File_Path    VARCHAR2(300) );
```

4.1.10 JUDGMENT Table

```
CREATE TABLE JUDGMENT (     Judgment_ID    NUMBER(8)      CONSTRAINT pk_judgment PRIMARY KEY,
Case_ID        NUMBER(8)      CONSTRAINT fk_judg_case  REFERENCES CASE_INFO(Case_ID)
CONSTRAINT uq_judg_case UNIQUE,     Judge_ID       NUMBER(5)      CONSTRAINT fk_judg_judge
REFERENCES JUDGE(Judge_ID),     Judgment_Date  DATE           NOT NULL,     Decision
VARCHAR2(100)  NOT NULL,     Summary        VARCHAR2(1000) );
```

4.2 Sequences for Primary Keys

```
CREATE SEQUENCE seq_court     START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE
seq_judge     START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE seq_users
START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE seq_case      START WITH 1
INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE seq_party     START WITH 1 INCREMENT BY 1
NOCACHE NOCYCLE; CREATE SEQUENCE seq_lawyer    START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE SEQUENCE seq_rep       START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE
seq_hearing   START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE seq_document
START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE; CREATE SEQUENCE seq_judgment  START WITH 1
INCREMENT BY 1 NOCACHE NOCYCLE;
```

4.3 Index Creation Statements

Indexes are created on frequently queried and filtered columns to improve retrieval performance:

```
-- CASE_INFO indexes CREATE INDEX idx_case_status      ON CASE_INFO(Status); CREATE INDEX
idx_case_type        ON CASE_INFO(Case_Type); CREATE INDEX idx_case_court       ON
CASE_INFO(Court_ID); CREATE INDEX idx_case_judge       ON CASE_INFO(Judge_ID); CREATE INDEX
idx_case_filing_date ON CASE_INFO(Filing_Date);  -- PARTY indexes CREATE INDEX idx_party_name
ON PARTY(Full_Name); CREATE INDEX idx_party_case       ON PARTY(Case_ID);  -- HEARING indexes
CREATE INDEX idx_hearing_date     ON HEARING(Hearing_Date); CREATE INDEX idx_hearing_case
ON HEARING(Case_ID);  -- DOCUMENT indexes CREATE INDEX idx_doc_type         ON
DOCUMENT(Doc_Type); CREATE INDEX idx_doc_case         ON DOCUMENT(Case_ID); CREATE INDEX
```

```
idx_doc_upload_date  ON DOCUMENT(Upload_Date);  -- LAWYER indexes CREATE INDEX
idx_lawyer_name      ON LAWYER(Full_Name);  -- JUDGMENT indexes CREATE INDEX
idx_judgment_date    ON JUDGMENT(Judgment_Date);  -- REPRESENTATION indexes CREATE INDEX
idx_rep_case         ON REPRESENTATION(Case_ID); CREATE INDEX idx_rep_lawyer       ON
REPRESENTATION(Lawyer_ID);
```

4.4 Justification of Index Selection

Index	Column(s)	Justification
idx_case_status	CASE_INFO.Status	Most common filter: users search by Pending/Decided status
idx_case_filing_date	CASE_INFO.Filing_Date	Date-range queries for reports are very frequent
idx_party_name	PARTY.Full_Name	Party name search is the primary lookup operation
idx_hearing_date	HEARING.Hearing_Date	Cause lists require fast date-based hearing retrieval
idx_doc_type	DOCUMENT.Doc_Type	Document filtering by type (FIR, Petition) is common
idx_rep_case	REPRESENTATION.Case_ID	Joining REPRESENTATION to CASE_INFO is a core operation